Control Plane

AXION Control Plane Build Outline

0) Definition

Goal: a repo-native “referee + recorder” that enforces AXION gates, runs verification, and keeps memory fresh **without depending on any one agent**.

---

1) Control Plane Scope (MVP)

1.1 Must-do capabilities

- Load AXION project state (registries, manifests, ledgers)
- Compute “next action” from ORD/PLAN rules
- Dispatch work to an executor (agent or human) via **work units**
- Run verification commands and capture outputs as proof
- Evaluate gates (pass/fail) using Gate DSL rules
- Write back updated state to repo (indexes, proofs, reports)

1.2 Explicit non-goals (for MVP)

- No autonomous coding inside the control plane (agents do that)
- No complex UI (CLI + JSON reports only)
- No multi-repo orchestration (single repo)

---

2) Repo Contracts (Files the Control Plane Owns)

Create a dedicated folder, e.g. `/axion/`.

2.1 State files (source of truth)

- `/axion/manifest.json` — project/version stamps + artifact inventory
- `/axion/spec_index.json` — list of spec items (screen/endpoint/entity/etc.)
- `/axion/ledger.json` — completed work units + proof pointers
- `/axion/gates.json` — gate definitions + status history
- `/axion/standards_index.json` — standards registry (IDs, applicability)

2.2 Run outputs (generated each run)

- `/axion/runs/<run_id>/run_log.jsonl` — step-by-step trace
- `/axion/runs/<run_id>/gate_report.json` — pass/fail + reasons
- `/axion/runs/<run_id>/proof_bundle.json` — captured evidence references

2.3 Work queue (agent-agnostic)

- `/axion/queue/PLAN_UNIT_<id>.json` — work unit assignments
- `/axion/queue/RESULT_<id>.json` — executor result (diff refs + notes)

---

3) Core Components (Code Modules)

3.1 Loader

- Reads all AXION state files
- Validates schemas (hard fail if invalid)
- Creates an in-memory “ProjectState” object

3.2 Planner (Next-Action Engine)

Inputs: ProjectState + ORD/PLAN rules
Outputs: one of:

- `CreateWorkUnit`
- `RunVerification`
- `UpdateIndexes`
- `Block` (missing required inputs/proof)

Rules it must support:

- dependency ordering (ORD graph)
- coverage checks (SPEC_INDEX category coverage)
- “missing artifact” detection
- “missing proof” detection

3.3 Dispatcher (Executor Bridge)

Two MVP dispatch modes:

- **File-based:** writes `PLAN_UNIT.json` into `/axion/queue/`

- **CLI prompt mode:** prints the unit + expected outputs

Later adapters (optional):

- Replit Agent adapter
- GitHub Issues adapter
- LangGraph/CrewAI adapter

3.4 Verifier (Tool Runner)

- Runs configured commands: `typecheck`, `lint`, `test`, `build`, etc.
- Captures outputs to `/axion/runs/<run_id>/`
- Normalizes results into structured “Evidence” objects

3.5 Gate Engine (Policy Enforcement)

- Loads Gate DSL rules
- Evaluates:
  - artifact existence/non-empty
  - schema validity
  - SPEC_INDEX coverage targets
  - ledger required fields (implementation_refs, verification_proof)
  - upstream locks (if you have dependency packs)
- Produces `gate_report.json` with:
  - gate status
  - failure reasons
  - remediation checklist

3.6 Writer (State Committer)

- Updates ledger/spec index/manifest
- Writes run artifacts
- Optionally commits changes to git with standard message

---

4) Gate DSL MVP (Minimal)

Start with a tiny set of predicates:

- `has_artifact(path)`
- `non_empty(path)`
- `coverage(category) == 100`

- `ledger.require_fields([...])`
- `commands.pass(["lint","test","build"])`

Output must always include:

- gate name
- predicate results
- actionable missing items list

---

5) CLI Surface (First UX)

Implement these commands:

5.1 `axion init`

- scaffolds `/axion/` folder + base files + schemas

5.2 `axion run`

- executes one safe “next action” (planner decides)

5.3 `axion cycle`

- loops: plan → dispatch → verify → gate → write (until block/fail)

5.4 `axion verify`

- runs verifier + gate engine only
- outputs:
  - terminal summary
  - `/axion/runs/<run_id>/gate_report.json`

5.5 `axion status`

- prints current stage, gate status, queue size, last failures

---

6) Evidence & Memory Capture (Your “fresh repo memory” bot)

Define a **Proof Schema** (structured, not chat dumps):

- `proof_id`
- `type` (test_output, build_log, screenshot, manual_check, diff_ref)
- `source` (command, file, url, commit)
- `hash` (optional but ideal)
- `applies_to` (spec item IDs)
- `notes`

Capture automatically:

- command outputs
- git diff stats
- changed files list
- failing test names
- build artifacts metadata

---

7) Integration Triggers (When it runs)

7.1 Local

- developer runs CLI

7.2 CI (PR check)

- run `axion verify` on every PR
- fail PR if gates fail

7.3 Merge/Releases

- run `axion pack` (later) to produce bundle/manifest stamp

---

8) Golden Path MVP (First end-to-end demo)

Pick one vertical slice:

- 1 domain
- 1–2 endpoints
- 1 entity

- 1 screen
- tests + lint + build

Requirement:

- can run `axion cycle` and end with a **passing gate report** and updated ledger.

---

9) Hardening Phase (After MVP works once)

- determinism: stable run IDs, stable output ordering
- concurrency: lock files to avoid parallel runs clobbering state
- rollback: if verify fails, don't commit state changes
- audit trail: append-only history for gate results

---

10) Deliverables Checklist

By the end you should have:

- `/axion/` folder structure in repo
- JSON schemas for state + proofs + gate reports
- working CLI with `verify/status/run`
- gate engine producing deterministic pass/fail reports
- one Golden Path project proving the system

Repo folder tree

Below is a **copy-pasteable starter kit**: folder tree, **JSON schema drafts**, and a **minimal TypeScript control-plane CLI skeleton** (agent-agnostic, repo-native).

---

1) Repo folder tree (authoritative)

```
/axion/
  manifest.json
  spec_index.json
  ledger.json
  gates.json
  standards_index.json
  config.json
  queue/
    .gitkeep
  runs/
    .gitkeep
  schemas/
    manifest.schema.json
    spec_index.schema.json
    ledger.schema.json
    gates.schema.json
    proof.schema.json
    gate_report.schema.json
    plan_unit.schema.json
    result.schema.json

/tools/axion-control-plane/
  package.json
  tsconfig.json
  src/
    bin.ts
    cli.ts
    config.ts
    state/
      loadState.ts
      writeState.ts
      types.ts
      paths.ts
    planner/
      nextAction.ts
    verifier/
```

```
    runCommands.ts
    normalize.ts
  gates/
    evaluateGates.ts
    predicates.ts
  queue/
    emitPlanUnit.ts
    readResult.ts
  util/
    exec.ts
    fs.ts
    hash.ts
    time.ts
```

> You can also place `/tools/axion-control-plane` at repo root if you prefer; keeping it under `/tools` is clean.

---

2) AXION state files (starter contents)

`/axion/config.json`

```
{
  "project": {
    "name": "axion-project",
    "version": "0.1.0"
  },
  "paths": {
    "axionRoot": "axion",
    "queueDir": "axion/queue",
    "runsDir": "axion/runs"
  },
  "verifyCommands": [
    { "id": "typecheck", "cmd": "npm run typecheck", "required": true },
    { "id": "lint", "cmd": "npm run lint", "required": true },
    { "id": "test", "cmd": "npm test", "required": true },
    { "id": "build", "cmd": "npm run build", "required": true }
  ],
  "gates": {
    "active": ["GATE-BASELINE", "GATE-COVERAGE", "GATE-VERIFY"]
  }
}
```

/axion/manifest.json


```
{
  "schema_version": "1.0.0",
  "project": { "name": "axion-project", "version": "0.1.0" },
  "generated_at": "2026-02-24T00:00:00Z",
  "artifacts": [],
  "last_run": null
}
```


/axion/spec_index.json


```
{
  "schema_version": "1.0.0",
  "items": []
}
```


/axion/ledger.json


```
{
  "schema_version": "1.0.0",
  "work_units": [],
  "proofs": []
}
```


/axion/gates.json


```
{
  "schema_version": "1.0.0",
  "definitions": [
    {
      "id": "GATE-BASELINE",
      "title": "Baseline AXION files exist",
      "predicates": [
        { "op": "has_file", "path": "axion/manifest.json" },
        { "op": "has_file", "path": "axion/spec_index.json" },
        { "op": "has_file", "path": "axion/ledger.json" },
        { "op": "has_file", "path": "axion/config.json" }
      ]
    },
    {
```

```
      "id": "GATE-COVERAGE",
      "title": "SPEC_INDEX coverage targets",
      "predicates": [
        { "op": "coverage_at_least", "category": "endpoint", "pct": 0 },
        { "op": "coverage_at least", "category": "entity", "pct": 0 }
      ]
    },
    {
      "id": "GATE-VERIFY",
      "title": "Verification commands must pass",
      "predicates": [
        { "op": "commands_pass", "ids": ["typecheck", "lint", "test", "build"] }
      ]
    }
  ],
  "history": []
}
```

/axion/standards_index.json

```
{
  "schema_version": "1.0.0",
  "standards": []
}
```

---

3) JSON Schemas (drafts you can enforce)

These are “good enough” to start enforcing structure. Tighten later.

/axion/schemas/plan_unit.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "axion://schemas/plan_unit.schema.json",
  "type": "object",
  "required": ["schema_version", "unit_id", "title", "created_at", "targets", "acceptance",
"outputs_expected"],
  "properties": {
    "schema_version": { "type": "string" },
    "unit_id": { "type": "string" },
    "title": { "type": "string" },
```

```
    "created_at": { "type": "string" },
    "priority": { "type": "integer", "minimum": 0, "maximum": 5 },
    "targets": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["category", "id"],
        "properties": {
          "category": { "type": "string", "enum":
["screen","endpoint","entity","business_rule","component","acceptance_criterion","test_case","flo
w"] },
          "id": { "type": "string" }
        }
      }
    },
    "acceptance": {
      "type": "array",
      "items": { "type": "string" }
    },
    "outputs_expected": {
      "type": "array",
      "items": { "type": "string" }
    },
    "notes": { "type": "string" }
  },
  "additionalProperties": false
}
```

/axion/schemas/result.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "axion://schemas/result.schema.json",
  "type": "object",
  "required": ["schema_version", "unit_id", "submitted_at", "implementation_refs", "proof_refs"],
  "properties": {
    "schema_version": { "type": "string" },
    "unit_id": { "type": "string" },
    "submitted_at": { "type": "string" },
    "implementation_refs": {
      "type": "array",
      "items": { "type": "string" }
    },
    "proof_refs": {
```

```
      "type": "array",
      "items": { "type": "string" }
    },
    "summary": { "type": "string" }
  },
  "additionalProperties": false
}
```

/axion/schemas/proof.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "axion://schemas/proof.schema.json",
  "type": "object",
  "required": ["schema_version", "proof_id", "type", "created_at", "source", "applies_to"],
  "properties": {
    "schema_version": { "type": "string" },
    "proof_id": { "type": "string" },
    "type": { "type": "string", "enum":
["test_output","build_log","lint_log","typecheck_log","manual_check","diff_ref","artifact_ref"] },
    "created_at": { "type": "string" },
    "source": {
      "type": "object",
      "required": ["kind"],
      "properties": {
        "kind": { "type": "string", "enum": ["command","file","url","commit"] },
        "value": { "type": "string" }
      },
      "additionalProperties": false
    },
    "hash": { "type": "string" },
    "applies_to": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["category", "id"],
        "properties": {
          "category": { "type": "string" },
          "id": { "type": "string" }
        }
      }
    },
    "notes": { "type": "string" }
  },
```

```
  "additionalProperties": false
}
```

/axion/schemas/gate_report.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "axion://schemas/gate_report.schema.json",
  "type": "object",
  "required": ["schema_version", "run_id", "created_at", "overall", "gates"],
  "properties": {
    "schema_version": { "type": "string" },
    "run_id": { "type": "string" },
    "created_at": { "type": "string" },
    "overall": { "type": "string", "enum": ["PASS","FAIL"] },
    "gates": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["gate_id", "title", "status", "predicate_results"],
        "properties": {
          "gate_id": { "type": "string" },
          "title": { "type": "string" },
          "status": { "type": "string", "enum": ["PASS","FAIL"] },
          "predicate_results": {
            "type": "array",
            "items": {
              "type": "object",
              "required": ["op", "status"],
              "properties": {
                "op": { "type": "string" },
                "status": { "type": "string", "enum": ["PASS","FAIL"] },
                "detail": { "type": "string" }
              },
              "additionalProperties": false
            }
          },
          "remediation": { "type": "array", "items": { "type": "string" } }
        },
        "additionalProperties": false
      }
    },
    "command_results": {
      "type": "array",
```

```
      "items": {
        "type": "object",
        "required": ["id","status","duration_ms"],
        "properties": {
          "id": { "type": "string" },
          "status": { "type": "string", "enum": ["PASS","FAIL","SKIP"] },
          "duration_ms": { "type": "integer", "minimum": 0 },
          "log_path": { "type": "string" }
        },
        "additionalProperties": false
      }
    }
  },
  "additionalProperties": false
}
```

(You can add schemas for `manifest/spec_index/ledger/gates` later; MVP is fine with the above + basic JSON parsing.)

---

4) Minimal TypeScript Control Plane (working skeleton)

`/tools/axion-control-plane/package.json`

```
{
  "name": "axion-control-plane",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "bin": {
    "axion": "dist/bin.js"
  },
  "scripts": {
    "build": "tsc -p tsconfig.json",
    "dev": "node --enable-source-maps dist/bin.js",
    "axion:build": "npm run build && node dist/bin.js status",
    "lint": "echo \"(add lint)\"",
    "typecheck": "echo \"(add typecheck)\"",
    "test": "echo \"(add tests)\""
  },
  "dependencies": {
    "zod": "^3.25.0"
```

```
  },
  "devDependencies": {
    "typescript": "^5.6.0"
  }
}
```

/tools/axion-control-plane/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ES2022",
    "moduleResolution": "Bundler",
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true
  },
  "include": ["src"]
}
```

/tools/axion-control-plane/src/bin.ts

```
#!/usr/bin/env node
import { runCli } from "./cli.js";

runCli(process.argv.slice(2)).catch((err) => {
  console.error(err?.message ?? err);
  process.exit(1);
});
```

/tools/axion-control-plane/src/cli.ts

```
import { loadConfig } from "./config.js";
import { loadState, writeManifestLastRun } from "./state/loadState.js";
import { nextAction } from "./planner/nextAction.js";
import { runVerifyCommands } from "./verifier/runCommands.js";
import { evaluateGates } from "./gates/evaluateGates.js";
import { emitPlanUnit } from "./queue/emitPlanUnit.js";
import { nowIso, newRunId } from "./util/time.js";
```

```
import path from "node:path";

type Cmd = "status" | "verify" | "run" | "init";

export async function runCli(args: string[]) {
  const cmd = (args[0] as Cmd) ?? "status";
  const repoRoot = process.cwd();

  if (cmd === "init") {
    console.log("Init is repo-specific (scaffold /axion manually or add a scaffold command later).");
    return;
  }

  const config = await loadConfig(repoRoot);
  const state = await loadState(repoRoot, config);

  if (cmd === "status") {
    console.log(`AXION Control Plane`);
    console.log(`Project: ${config.project.name} v${config.project.version}`);
    console.log(`Last run: ${state.manifest.last_run ?? "none"}`);
    console.log(`Queue dir: ${config.paths.queueDir}`);
    return;
  }

  const runId = newRunId();
  const runDir = path.join(repoRoot, config.paths.runsDir, runId);

  if (cmd === "verify") {
    const commandResults = await runVerifyCommands(repoRoot, runDir,
config.verifyCommands);
    const report = await evaluateGates(repoRoot, runId, nowIso(), state, config,
commandResults);

    await writeManifestLastRun(repoRoot, config, runId);
    // Write gate report
    await state.io.writeJson(path.join(runDir, "gate_report.json"), report);

    const ok = report.overall === "PASS";
    console.log(ok ? "PASS" : "FAIL");
    if (!ok) process.exit(2);
    return;
  }

  if (cmd === "run") {
```

```
    const action = await nextAction(repoRoot, state, config);

    if (action.kind === "BLOCK") {
      console.log("BLOCKED:");
      for (const r of action.reasons) console.log(`- ${r}`);
      return;
    }

    if (action.kind === "EMIT_PLAN_UNIT") {
      const planPath = await emitPlanUnit(repoRoot, config, action.unit);
      console.log(`PLAN_UNIT emitted: ${planPath}`);
      return;
    }

    if (action.kind === "VERIFY") {
      const commandResults = await runVerifyCommands(repoRoot, runDir,
config.verifyCommands);
      const report = await evaluateGates(repoRoot, runId, nowIso(), state, config,
commandResults);

      await writeManifestLastRun(repoRoot, config, runId);
      await state.io.writeJson(path.join(runDir, "gate_report.json"), report);

      console.log(report.overall);
      if (report.overall !== "PASS") process.exit(2);
      return;
    }

    // Fallback
    console.log(`Unknown action: ${(action as any).kind}`);
    return;
  }

  console.log(`Unknown command: ${cmd}`);
  console.log(`Use: axion status | axion verify | axion run | axion init`);
}
```

/tools/axion-control-plane/src/config.ts

```
import { z } from "zod";
import { readJson } from "./util/fs.js";
import path from "node:path";

const VerifyCommand = z.object({
```

```
  id: z.string(),
  cmd: z.string(),
  required: z.boolean().default(true)
});

const ConfigSchema = z.object({
  project: z.object({
    name: z.string(),
    version: z.string()
  }),
  paths: z.object({
    axionRoot: z.string(),
    queueDir: z.string(),
    runsDir: z.string()
  }),
  verifyCommands: z.array(VerifyCommand),
  gates: z.object({
    active: z.array(z.string())
  })
});

export type AxionConfig = z.infer<typeof ConfigSchema>;
export type VerifyCommand = z.infer<typeof VerifyCommand>;

export async function loadConfig(repoRoot: string): Promise<AxionConfig> {
  const p = path.join(repoRoot, "axion", "config.json");
  const raw = await readJson(p);
  return ConfigSchema.parse(raw);
}
```

/tools/axion-control-plane/src/state/loadState.ts

```
import path from "node:path";
import { readJson, writeJson } from "../util/fs.js";
import type { AxionConfig } from "../config.js";

export type Manifest = {
  schema_version: string;
  project: { name: string; version: string };
  generated_at: string;
  artifacts: any[];
  last_run: string | null;
};
```

```
export type SpecIndex = { schema_version: string; items: any[] };
export type Ledger = { schema_version: string; work_units: any[]; proofs: any[] };
export type Gates = { schema_version: string; definitions: any[]; history: any[] };

export type LoadedState = {
  manifest: Manifest;
  specIndex: SpecIndex;
  ledger: Ledger;
  gates: Gates;
  io: {
    readJson: (p: string) => Promise<any>;
    writeJson: (p: string, v: any) => Promise<void>;
  };
};

export async function loadState(repoRoot: string, config: AxionConfig): Promise<LoadedState>
{
  const ax = (rel: string) => path.join(repoRoot, rel);

  const manifest = (await readJson(ax("axion/manifest.json"))) as Manifest;
  const specIndex = (await readJson(ax("axion/spec_index.json"))) as SpecIndex;
  const ledger = (await readJson(ax("axion/ledger.json"))) as Ledger;
  const gates = (await readJson(ax("axion/gates.json"))) as Gates;

  return {
    manifest,
    specIndex,
    ledger,
    gates,
    io: { readJson, writeJson }
  };
}

export async function writeManifestLastRun(repoRoot: string, config: AxionConfig, runId: string)
{
  const p = path.join(repoRoot, "axion/manifest.json");
  const manifest = (await readJson(p)) as Manifest;
  manifest.last_run = runId;
  manifest.generated_at = new Date().toISOString();
  await writeJson(p, manifest);
}
```

/tools/axion-control-plane/src/planner/nextAction.ts

```
import type { LoadedState } from "../state/loadState.js";
import type { AxionConfig } from "../config.js";

type BlockAction = { kind: "BLOCK"; reasons: string[] };
type VerifyAction = { kind: "VERIFY" };
type EmitPlanUnitAction = { kind: "EMIT_PLAN_UNIT"; unit: any };

export type NextAction = BlockAction | VerifyAction | EmitPlanUnitAction;

/**
 * MVP planner:
 * - If queue is empty, emit a placeholder PLAN_UNIT.
 * - Otherwise, run verify.
 *
 * Replace this with your ORD/PLAN logic once you wire real work-unit generation.
 */
export async function nextAction(repoRoot: string, state: LoadedState, config: AxionConfig):
Promise<NextAction> {
  // Minimal “block if baseline missing” guard can be added here.
  const hasAnyItems = state.specIndex.items.length > 0;

  if (!hasAnyItems) {
    return {
      kind: "EMIT_PLAN_UNIT",
      unit: {
        schema_version: "1.0.0",
        unit_id: `UNIT-${Date.now()}`,
        title: "Seed SPEC_INDEX with initial items (MVP)",
        created_at: new Date().toISOString(),
        priority: 3,
        targets: [{ category: "endpoint", id: "EP-0001" }],
        acceptance: ["spec_index.json contains at least 1 endpoint item with id EP-0001"],
        outputs_expected: ["axion/spec_index.json update"],
        notes: "Executor: add the first SPEC_INDEX item(s) so coverage can be measured."
      }
    };
  }

  return { kind: "VERIFY" };
}
```

/tools/axion-control-plane/src/verifier/runCommands.ts

```
import path from "node:path";
```

```
import { ensureDir, writeText } from "../util/fs.js";
import { execCmd } from "../util/exec.js";
import type { VerifyCommand } from "../config.js";

export type CommandResult = {
  id: string;
  status: "PASS" | "FAIL" | "SKIP";
  duration_ms: number;
  log_path?: string;
};

export async function runVerifyCommands(repoRoot: string, runDir: string, cmds:
VerifyCommand[]): Promise<CommandResult[]> {
  await ensureDir(runDir);

  const results: CommandResult[] = [];

  for (const c of cmds) {
    const start = Date.now();
    const logPath = path.join(runDir, `${c.id}.log`);

    try {
      const out = await execCmd(c.cmd, repoRoot);
      await writeText(logPath, out.combined);
      results.push({ id: c.id, status: "PASS", duration_ms: Date.now() - start, log_path: logPath });
    } catch (e: any) {
      const combined = e?.combined ?? e?.message ?? String(e);
      await writeText(logPath, combined);
      results.push({ id: c.id, status: "FAIL", duration_ms: Date.now() - start, log_path: logPath });
      if (c.required) {
        // Continue gathering logs, but keep failure recorded
      }
    }
  }

  return results;
}
```

/tools/axion-control-plane/src/gates/evaluateGates.ts

```
import path from "node:path";
import type { LoadedState } from "../state/loadState.js";
import type { AxionConfig } from "../config.js";
import type { CommandResult } from "../verifier/runCommands.js";
```

```
import { predicateHasFile, predicateCommandsPass, predicateCoverageAtLeast } from
"./predicates.js";

type Predicate =
  | { op: "has_file"; path: string }
  | { op: "commands_pass"; ids: string[] }
  | { op: "coverage_at_least"; category: string; pct: number };

export type GateReport = {
  schema_version: string;
  run_id: string;
  created_at: string;
  overall: "PASS" | "FAIL";
  gates: Array<{
    gate_id: string;
    title: string;
    status: "PASS" | "FAIL";
    predicate_results: Array<{ op: string; status: "PASS" | "FAIL"; detail?: string }>;
    remediation?: string[];
  }>;
  command_results?: Array<{ id: string; status: "PASS" | "FAIL" | "SKIP"; duration_ms: number;
log_path?: string }>;
};

export async function evaluateGates(
  repoRoot: string,
  runId: string,
  createdAt: string,
  state: LoadedState,
  config: AxionConfig,
  commandResults: CommandResult[]
): Promise<GateReport> {
  const active = new Set(config.gates.active);

  const gateDefs = state.gates.definitions.filter((g: any) => active.has(g.id));

  const gatesOut: GateReport["gates"] = [];

  for (const g of gateDefs) {
    const preds = (g.predicates ?? []) as Predicate[];
    const predResults: Array<{ op: string; status: "PASS" | "FAIL"; detail?: string }> = [];
    const remediation: string[] = [];

    for (const p of preds) {
```

```
      if (p.op === "has_file") {
        const ok = await predicateHasFile(repoRoot, p.path);
        predResults.push({ op: `has_file(${p.path})`, status: ok ? "PASS" : "FAIL" });
        if (!ok) remediation.push(`Create missing file: ${p.path}`);
      } else if (p.op === "commands_pass") {
        const ok = predicateCommandsPass(commandResults, p.ids);
        predResults.push({ op: `commands_pass(${p.ids.join(",")})`, status: ok ? "PASS" : "FAIL" });
        if (!ok) remediation.push(`Fix failing verify commands: ${p.ids.join(", ")}`);
      } else if (p.op === "coverage_at_least") {
        const ok = predicateCoverageAtLeast(state, p.category, p.pct);
        predResults.push({ op: `coverage_at_least(${p.category},${p.pct})`, status: ok ? "PASS" :
"FAIL" });
        if (!ok) remediation.push(`Increase SPEC_INDEX coverage for category "${p.category}" to
>= ${p.pct}%`);
      } else {
        predResults.push({ op: (p as any).op ?? "unknown", status: "FAIL", detail: "Unknown
predicate op" });
        remediation.push(`Unsupported predicate: ${(p as any).op}`);
      }
    }

    const gatePass = predResults.every((r) => r.status === "PASS");
    gatesOut.push({
      gate_id: g.id,
      title: g.title ?? g.id,
      status: gatePass ? "PASS" : "FAIL",
      predicate_results: predResults,
      remediation: remediation.length ? remediation : undefined
    });
  }

  const overall = gatesOut.every((g) => g.status === "PASS") ? "PASS" : "FAIL";

  return {
    schema_version: "1.0.0",
    run_id: runId,
    created_at: createdAt,
    overall,
    gates: gatesOut,
    command_results: commandResults
  };
}
```

/tools/axion-control-plane/src/gates/predicates.ts

```
import fs from "node:fs/promises";
import path from "node:path";
import type { CommandResult } from "../verifier/runCommands.js";
import type { LoadedState } from "../state/loadState.js";

export async function predicateHasFile(repoRoot: string, relPath: string): Promise<boolean> {
  try {
    await fs.stat(path.join(repoRoot, relPath));
    return true;
  } catch {
    return false;
  }
}

export function predicateCommandsPass(results: CommandResult[], ids: string[]): boolean {
  const map = new Map(results.map((r) => [r.id, r.status]));
  return ids.every((id) => map.get(id) === "PASS");
}

/**
 * MVP coverage: percent = (#items in category) / max(1, #items total) * 100
 * Replace with your real coverage semantics once SPEC_INDEX has required targets per
category.
 */
export function predicateCoverageAtLeast(state: LoadedState, category: string, pct: number):
boolean {
  const total = Math.max(1, state.specIndex.items.length);
  const catCount = state.specIndex.items.filter((it: any) => it?.category === category).length;
  const computed = (catCount / total) * 100;
  return computed >= pct;
}
```

/tools/axion-control-plane/src/queue/emitPlanUnit.ts

```
import path from "node:path";
import type { AxionConfig } from "../config.js";
import { ensureDir, writeJson } from "../util/fs.js";

export async function emitPlanUnit(repoRoot: string, config: AxionConfig, unit: any):
Promise<string> {
  const dir = path.join(repoRoot, config.paths.queueDir);
  await ensureDir(dir);
```

```
  const file = `PLAN_UNIT_${unit.unit_id}.json`;
  const full = path.join(dir, file);
  await writeJson(full, unit);
  return full;
}
```

/tools/axion-control-plane/src/util/exec.ts

```
import { spawn } from "node:child_process";

export async function execCmd(cmd: string, cwd: string): Promise<{ code: number; combined:
string }> {
  return new Promise((resolve, reject) => {
    const child = spawn(cmd, { cwd, shell: true, stdio: ["ignore", "pipe", "pipe"] });

    let combined = "";

    child.stdout.on("data", (d) => (combined += d.toString()));
    child.stderr.on("data", (d) => (combined += d.toString()));

    child.on("close", (code) => {
      if (code === 0) resolve({ code: code ?? 0, combined });
      else reject({ code: code ?? 1, combined });
    });
    child.on("error", (err) => reject({ code: 1, combined: combined + "\n" + String(err) }));
  });
}
```

/tools/axion-control-plane/src/util/fs.ts

```
import fs from "node:fs/promises";
import path from "node:path";

export async function ensureDir(p: string) {
  await fs.mkdir(p, { recursive: true });
}

export async function readJson(p: string): Promise<any> {
  const txt = await fs.readFile(p, "utf8");
  return JSON.parse(txt);
}
```

```
export async function writeJson(p: string, v: any): Promise<void> {
  await ensureDir(path.dirname(p));
  await fs.writeFile(p, JSON.stringify(v, null, 2) + "\n", "utf8");
}

export async function writeText(p: string, txt: string): Promise<void> {
  await ensureDir(path.dirname(p));
  await fs.writeFile(p, txt, "utf8");
}
```

`/tools/axion-control-plane/src/util/time.ts`

```
export function nowIso(): string {
  return new Date().toISOString();
}

export function newRunId(): string {
  // deterministic enough for now; replace with ULID later if you want ordering guarantees
  return `RUN-${Date.now()}`;
}
```

---

5) How you run it (today)

From `/tools/axion-control-plane`:

```
npm i
npm run build
node dist/bin.js status
node dist/bin.js verify
node dist/bin.js run
```

---

6) What to wire next (the real “control plane” moment)

Once this skeleton is in:

1. Replace `planner/nextAction.ts` with your **ORD/PLAN logic** (read SPEC_INDEX + ledger → emit next PLAN_UNIT).

2. Add `queue/readResult.ts` to ingest `RESULT_*.json` and update `ledger.json` (implementation_refs + proof_refs).
3. Tighten coverage semantics: coverage should be measured **per required category targets**, not “% of total items.”

Schema

Below are the **remaining strict schemas** (`manifest/spec_index/ledger/gates`) **plus** an `axion ingest-results` command that reads `RESULT_*.json` from `/axion/queue`, updates `ledger.json`, and (optionally) marks the unit “done”.

---

1) Remaining JSON Schemas

`/axion/schemas/manifest.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "axion://schemas/manifest.schema.json",
  "type": "object",
  "required": ["schema_version", "project", "generated_at", "artifacts", "last_run"],
  "properties": {
    "schema_version": { "type": "string" },
    "project": {
      "type": "object",
      "required": ["name", "version"],
      "properties": {
        "name": { "type": "string" },
        "version": { "type": "string" }
      },
      "additionalProperties": false
    },
    "generated_at": { "type": "string" },
    "last_run": { "type": ["string", "null"] },
    "artifacts": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["id", "type", "path", "version"],
        "properties": {
          "id": { "type": "string" },
          "type": { "type": "string" },
          "path": { "type": "string" },
          "version": { "type": "string" },
          "hash": { "type": "string" }
        },
        "additionalProperties": false
      }
    }
```

```
  },
  "additionalProperties": false
}
```

/axion/schemas/spec_index.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "axion://schemas/spec_index.schema.json",
  "type": "object",
  "required": ["schema_version", "items"],
  "properties": {
    "schema_version": { "type": "string" },
    "items": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["category", "id", "title"],
        "properties": {
          "category": {
            "type": "string",
            "enum": [
              "screen",
              "endpoint",
              "entity",
              "business_rule",
              "component",
              "acceptance_criterion",
              "test_case",
              "flow"
            ]
          },
          "id": { "type": "string" },
          "title": { "type": "string" },
          "status": { "type": "string", "enum": ["draft", "ready", "implemented", "verified"], "default":
"draft" },
          "owner_pack": { "type": "string" },
          "source_refs": { "type": "array", "items": { "type": "string" } },
          "implementation_refs": { "type": "array", "items": { "type": "string" } },
          "verification_proof": { "type": "array", "items": { "type": "string" } }
        },
        "additionalProperties": false
      }
    }
```

```
  },
  "additionalProperties": false
}
```

/axion/schemas/ledger.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "axion://schemas/ledger.schema.json",
  "type": "object",
  "required": ["schema_version", "work_units", "proofs"],
  "properties": {
    "schema_version": { "type": "string" },
    "work_units": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["unit_id", "title", "status", "created_at"],
        "properties": {
          "unit_id": { "type": "string" },
          "title": { "type": "string" },
          "status": { "type": "string", "enum": ["queued", "in_progress", "done", "blocked"], "default":
"queued" },
          "created_at": { "type": "string" },
          "completed_at": { "type": ["string", "null"] },
          "targets": {
            "type": "array",
            "items": {
              "type": "object",
              "required": ["category", "id"],
              "properties": {
                "category": { "type": "string" },
                "id": { "type": "string" }
              },
              "additionalProperties": false
            }
          },
          "implementation_refs": { "type": "array", "items": { "type": "string" } },
          "verification_proof": { "type": "array", "items": { "type": "string" } },
          "notes": { "type": "string" }
        },
        "additionalProperties": false
      }
    },
```

```
    "proofs": {
      "type": "array",
      "items": { "$ref": "axion://schemas/proof.schema.json" }
    }
  },
  "additionalProperties": false
}
```

/axion/schemas/gates.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "axion://schemas/gates.schema.json",
  "type": "object",
  "required": ["schema_version", "definitions", "history"],
  "properties": {
    "schema_version": { "type": "string" },
    "definitions": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["id", "title", "predicates"],
        "properties": {
          "id": { "type": "string" },
          "title": { "type": "string" },
          "predicates": {
            "type": "array",
            "items": {
              "type": "object",
              "required": ["op"],
              "properties": {
                "op": { "type": "string" },
                "path": { "type": "string" },
                "ids": { "type": "array", "items": { "type": "string" } },
                "category": { "type": "string" },
                "pct": { "type": "number", "minimum": 0, "maximum": 100 }
              },
              "additionalProperties": true
            }
          }
        },
        "additionalProperties": false
      }
    },
```

```
    "history": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["run_id", "created_at", "overall"],
        "properties": {
          "run_id": { "type": "string" },
          "created_at": { "type": "string" },
          "overall": { "type": "string", "enum": ["PASS", "FAIL"] }
        },
        "additionalProperties": false
      }
    }
  },
  "additionalProperties": false
}
```

---

2) Add `axion ingest-results` command

A) Update CLI command list

Edit `/tools/axion-control-plane/src/cli.ts`:

1. Add `ingest-results` to the Cmd type:

```
type Cmd = "status" | "verify" | "run" | "init" | "ingest-results";
```

2. Import the ingest function near the top:

```
import { ingestResults } from "./queue/ingestResults.js";
```

3. Add a handler block (place it after `status` and before runId creation, or anywhere early):

```
 if (cmd === "ingest-results") {
    const out = await ingestResults(repoRoot, config);
    console.log(`Ingested results: ${out.ingested}`);
    console.log(`Ledger updated: ${out.ledgerUpdated ? "yes" : "no"}`);
    return;
  }
```

B) Create `/tools/axion-control-plane/src/queue/ingestResults.ts`

```
import path from "node:path";
import fs from "node:fs/promises";
import type { AxionConfig } from "../config.js";
import { readJson, writeJson } from "../util/fs.js";

type ResultFile = {
  schema_version: string;
  unit_id: string;
  submitted_at: string;
  implementation_refs: string[];
  proof_refs: string[];
  summary?: string;
};

export async function ingestResults(repoRoot: string, config: AxionConfig): Promise<{
  ingested: number;
  ledgerUpdated: boolean;
}> {
  const queueDir = path.join(repoRoot, config.paths.queueDir);
  const ledgerPath = path.join(repoRoot, "axion/ledger.json");

  // Read ledger
  const ledger = await readJson(ledgerPath) as {
    schema_version: string;
    work_units: any[];
    proofs: any[];
  };

  // Find RESULT_*.json files
  let entries: string[] = [];
  try {
    entries = await fs.readdir(queueDir);
  } catch {
    return { ingested: 0, ledgerUpdated: false };
  }

  const resultFiles = entries.filter((f) => f.startsWith("RESULT_") && f.endsWith(".json"));
  if (resultFiles.length === 0) return { ingested: 0, ledgerUpdated: false };
```

```
  let ingested = 0;

  for (const file of resultFiles) {
    const full = path.join(queueDir, file);
    const result = (await readJson(full)) as ResultFile;

    // Upsert work unit row in ledger
    const existingIdx = ledger.work_units.findIndex((wu) => wu.unit_id === result.unit_id);

    if (existingIdx === -1) {
      ledger.work_units.push({
        unit_id: result.unit_id,
        title: result.summary ?? `Result for ${result.unit_id}`,
        status: "done",
        created_at: result.submitted_at,
        completed_at: result.submitted_at,
        targets: [],
        implementation_refs: result.implementation_refs ?? [],
        verification_proof: result.proof_refs ?? [],
        notes: result.summary ?? ""
      });
    } else {
      const wu = ledger.work_units[existingIdx];
      wu.status = "done";
      wu.completed_at = result.submitted_at;

      wu.implementation_refs = unique([...(wu.implementation_refs ?? []),
...(result.implementation_refs ?? [])]);
      wu.verification_proof = unique([...(wu.verification_proof ?? []), ...(result.proof_refs ?? [])]);

      if (result.summary) wu.notes = wu.notes ? `${wu.notes}\n${result.summary}` :
result.summary;
    }

    // Archive the processed result file (keeps queue clean and preserves history)
    const archiveDir = path.join(repoRoot, "axion", "queue", "_processed");
    await fs.mkdir(archiveDir, { recursive: true });
    await fs.rename(full, path.join(archiveDir, file));

    ingested++;
  }

  await writeJson(ledgerPath, ledger);
```

```
  return { ingested, ledgerUpdated: true };
}

function unique(arr: string[]): string[] {
  return Array.from(new Set(arr.filter(Boolean)));
}
```

---

3) (Optional but recommended) Auto-update SPEC_INDEX proof/impl refs

If you want AXION to stay “fresh” automatically, add this behavior inside `ingestResults`:

- For each `implementation_ref` and `proof_ref`, attach them to any SPEC_INDEX items targeted by that unit (requires your `PLAN_UNIT` to include `targets`, and you store `targets` in ledger work_units).
- That’s a small second pass once you wire real PLAN units.

---

4) How you use it right now

1. Executor/agent finishes a unit and writes:
- `/axion/queue/RESULT_UNIT-123.json`
2. Run:

```
node dist/bin.js ingest-results
```

3. Ledger updates, result file gets archived, `axion verify` can now enforce “ledger required fields” gates later.

---

predicate

Below is the **`ledger.require_fields(...)` predicate** wired end-to-end:

- add a new gate op: `ledger_require_fields`
- implement predicate evaluation
- update your sample `gates.json` to use it

---

1) Update `/axion/gates.json` (add a new gate)

Add this new gate definition (or append to `definitions`):

```
{
  "id": "GATE-LEDGER",
  "title": "Ledger done units must include implementation + proof",
  "predicates": [
    {
      "op": "ledger_require_fields",
      "status": "done",
      "fields": ["implementation_refs", "verification_proof"]
    }
  ]
}
```

Then update `/axion/config.json` to activate it:

```
"gates": {
  "active": ["GATE-BASELINE", "GATE-COVERAGE", "GATE-VERIFY", "GATE-LEDGER"]
}
```

---

2) Extend gate predicate types

Edit `/tools/axion-control-plane/src/gates/evaluateGates.ts`

A) Extend the `Predicate` union

Replace the current `Predicate` union with this:

```
type Predicate =
```

```
  | { op: "has_file"; path: string }
  | { op: "commands_pass"; ids: string[] }
  | { op: "coverage_at_least"; category: string; pct: number }
  | { op: "ledger_require_fields"; status: "done" | "queued" | "in_progress" | "blocked"; fields:
string[] };
```

B) Import the new predicate function

At the top, add:

```
import { predicateLedgerRequireFields } from "./predicates.js";
```

C) Add evaluation branch

Inside the `for (const p of preds)` loop, add this `else if` block:

```
    } else if (p.op === "ledger_require_fields") {
      const res = predicateLedgerRequireFields(state, p.status, p.fields);

      predResults.push({
        op: `ledger_require_fields(status=${p.status}, fields=${p.fields.join("|")})`,
        status: res.ok ? "PASS" : "FAIL",
        detail: res.ok ? undefined : res.detail
      });

      if (!res.ok) {
        remediation.push(
          `Fix ledger: all work_units with status="${p.status}" must include fields: ${p.fields.join(",
")}`
        );
        remediation.push(...res.remediation);
      }
```

---

3) Implement the predicate

Edit `/tools/axion-control-plane/src/gates/predicates.ts` and append:

```
export function predicateLedgerRequireFields(
  state: LoadedState,
  status: "done" | "queued" | "in_progress" | "blocked",
```

```
  fields: string[]
): { ok: boolean; detail?: string; remediation: string[] } {
  const remediation: string[] = [];

  const units = (state.ledger.work_units ?? []).filter((wu: any) => wu?.status === status);

  const missing: Array<{ unit_id: string; field: string }> = [];

  for (const wu of units) {
    for (const f of fields) {
      const v = (wu as any)[f];

      // field must exist and be non-empty array
      const ok = Array.isArray(v) ? v.length > 0 : Boolean(v);
      if (!ok) missing.push({ unit_id: wu.unit_id ?? "(unknown)", field: f });
    }
  }

  if (missing.length === 0) return { ok: true, remediation };

  // Build an actionable remediation list (cap to avoid spam)
  const capped = missing.slice(0, 25);
  for (const m of capped) {
    remediation.push(`- work_unit ${m.unit_id}: add ${m.field}`);
  }
  if (missing.length > capped.length) remediation.push(`- …and ${missing.length -
capped.length} more`);

  return {
    ok: false,
    detail: `${missing.length} missing field occurrences across ${units.length} "${status}"
work_units`,
    remediation
  };
}
```

---

4) Result (what you get)

Now `axion verify` will FAIL if any `ledger.work_units[]` with `status:"done"` is missing:

- `implementation_refs` (non-empty)
- `verification_proof` (non-empty)

…and the gate report will include an explicit list of which work units are missing what.

---

5) One small improvement you should do next (no extra fluff)

In `ingestResults.ts`, you currently write:

- `verification_proof: result.proof_refs`
- `implementation_refs: result.implementation_refs`

So this gate will pass **only if your executors actually fill those arrays**. That's correct and is exactly the enforcement you want.

Second Predicate

Below is `ledger_targets_implemented` wired end-to-end so a **done work unit** is only "valid" if its **targets** have:

- **implementation_refs** (non-empty)
- **verification_proof** (non-empty)

…and it prints exactly which spec items are missing what.

---

1) Update `/axion/gates.json` (add the new gate)

Add this to `definitions`:

```
{
  "id": "GATE-TARGETS",
  "title": "Done units must attach implementation + proof to targeted SPEC_INDEX items",
  "predicates": [
    {
      "op": "ledger_targets_implemented",
      "status": "done",
      "require": ["implementation_refs", "verification_proof"]
    }
  ]
}
```

Activate it in `/axion/config.json`:

```
"gates": {
  "active": ["GATE-BASELINE", "GATE-COVERAGE", "GATE-VERIFY", "GATE-LEDGER",
"GATE-TARGETS"]
}
```

---

2) Extend predicate types + evaluation

Edit `/tools/axion-control-plane/src/gates/evaluateGates.ts`

A) Extend the `Predicate` union

Add this line:

```
 | { op: "ledger_targets_implemented"; status: "done" | "queued" | "in_progress" | "blocked";
require: string[] };
```

So the union becomes:

```
type Predicate =
  | { op: "has_file"; path: string }
  | { op: "commands_pass"; ids: string[] }
  | { op: "coverage_at_least"; category: string; pct: number }
  | { op: "ledger_require_fields"; status: "done" | "queued" | "in_progress" | "blocked"; fields:
string[] }
  | { op: "ledger_targets_implemented"; status: "done" | "queued" | "in_progress" | "blocked";
require: string[] };
```

B) Import the new predicate function

At top:

```
import { predicateLedgerTargetsImplemented } from "./predicates.js";
```

C) Add evaluation branch

Inside `for (const p of preds)`:

```
    } else if (p.op === "ledger_targets_implemented") {
      const res = predicateLedgerTargetsImplemented(state, p.status, p.require);

      predResults.push({
        op: `ledger_targets_implemented(status=${p.status}, require=${p.require.join("|")})`,
        status: res.ok ? "PASS" : "FAIL",
        detail: res.ok ? undefined : res.detail
      });

      if (!res.ok) {
        remediation.push(
          `Fix SPEC_INDEX: for all targets in "${p.status}" work_units, require: ${p.require.join(",
")}`
        );
        remediation.push(...res.remediation);
      }
```

3) Implement predicate in `predicates.ts`

Edit `/tools/axion-control-plane/src/gates/predicates.ts` and append:

```
export function predicateLedgerTargetsImplemented(
  state: LoadedState,
  status: "done" | "queued" | "in_progress" | "blocked",
  require: string[]
): { ok: boolean; detail?: string; remediation: string[] } {
  const remediation: string[] = [];

  const workUnits = (state.ledger.work_units ?? []).filter((wu: any) => wu?.status === status);

  // Build quick lookup: "category:id" -> spec item
  const specMap = new Map<string, any>();
  for (const it of state.specIndex.items ?? []) {
    if (!it?.category || !it?.id) continue;
    specMap.set(`${it.category}:${it.id}`, it);
  }

  const missing: Array<{
    unit_id: string;
    target_key: string;
    missing_field: string;
  }> = [];

  for (const wu of workUnits) {
    const targets = wu.targets ?? [];
    for (const t of targets) {
      const key = `${t.category}:${t.id}`;
      const spec = specMap.get(key);

      if (!spec) {
        missing.push({
          unit_id: wu.unit_id ?? "(unknown)",
          target_key: key,
          missing_field: "SPEC_INDEX item missing"
        });
        continue;
      }
```

```
      for (const f of require) {
        const v = (spec as any)[f];
        const ok = Array.isArray(v) ? v.length > 0 : Boolean(v);
        if (!ok) {
          missing.push({
            unit_id: wu.unit_id ?? "(unknown)",
            target_key: key,
            missing_field: f
          });
        }
      }
    }
  }

  if (missing.length === 0) return { ok: true, remediation };

  // Actionable remediation list (cap)
  const capped = missing.slice(0, 25);
  for (const m of capped) {
    if (m.missing_field === "SPEC_INDEX item missing") {
      remediation.push(`- unit ${m.unit_id}: add SPEC_INDEX item for target ${m.target_key}`);
    } else {
      remediation.push(`- unit ${m.unit_id}: target ${m.target_key} add ${m.missing_field}`);
    }
  }
  if (missing.length > capped.length) remediation.push(`- …and ${missing.length -
capped.length} more`);

  return {
    ok: false,
    detail: `${missing.length} missing target-field occurrences across ${workUnits.length}
"${status}" work_units`,
    remediation
  };
}
```

---

4) Make it actually pass: attach refs to SPEC_INDEX automatically (recommended)

Right now, your `ingestResults.ts` updates the **ledger**, but doesn't propagate refs to SPEC_INDEX targets. Add this small upgrade so `GATE-TARGETS` can pass without manual edits.

A) Update `ingestResults.ts` to also update `spec_index.json`

Edit `/tools/axion-control-plane/src/queue/ingestResults.ts`

1. Load spec_index near the top:

```
const specIndexPath = path.join(repoRoot, "axion/spec_index.json");
const specIndex = await readJson(specIndexPath) as { schema_version: string; items: any[] };
```

2. When you upsert the work unit, ensure it has `targets` (if you emitted them in PLAN_UNIT, store them; for now it may be empty). If you eventually store targets in ledger work_units, use that. For MVP, you can do:

- If ledger work_unit has `targets`, use them.
- Else do nothing.

3. After updating the ledger work unit, propagate refs:

```
// Propagate refs to SPEC_INDEX items if the work_unit has targets
const workUnit = ledger.work_units.find((wu: any) => wu.unit_id === result.unit_id);
const targets = workUnit?.targets ?? [];

if (targets.length > 0) {
  // Build lookup
  const map = new Map(specIndex.items.map((it: any) => [`${it.category}:${it.id}`, it]));

  for (const t of targets) {
    const key = `${t.category}:${t.id}`;
    const spec = map.get(key);
    if (!spec) continue;

    spec.implementation_refs = unique([...(spec.implementation_refs ?? []),
...(result.implementation_refs ?? [])]);
    spec.verification_proof = unique([...(spec.verification_proof ?? []), ...(result.proof_refs ??
[])]);

    // Optional: bump status
    if ((spec.implementation_refs?.length ?? 0) > 0 && (spec.verification_proof?.length ?? 0) >
0) {
      spec.status = "verified";
    } else if ((spec.implementation_refs?.length ?? 0) > 0) {
```

```
      spec.status = "implemented";
    }
  }
}
```

4. At end, write spec_index too:

```
await writeJson(specIndexPath, specIndex);
```

> This requires that **targets are preserved on the ledger work_unit**. Your planner should emit PLAN_UNIT targets, and your ingest should store them into ledger work_units. Once that's true, this becomes automatic.

---

5) What this gives you (behavior)

- A work unit can be "done," but if it doesn't attach proof/impl to the SPEC_INDEX targets, **the repo fails gates**.
- That's the drift-killer: you can't "finish work" without leaving traceable evidence on the spec objects themselves.

If you want, I can now add the missing piece: `axion emit` (creates PLAN_UNITs from SPEC_INDEX gaps) + `axion accept` (marks a unit in_progress/done and writes RESULT template), so the loop is fully smooth for any agent.

axion emit

Below is a **drop-in** implementation for two new commands:

- **`axion emit`** → creates `PLAN_UNIT_*.json` from SPEC_INDEX "gaps"
- **`axion accept <UNIT_ID>`** → marks the unit `in_progress` in `ledger.json` and writes a **RESULT template** (`RESULT_<UNIT_ID>.json`) for the executor/agent to fill

This keeps AXION **agent-agnostic** and repo-native.

---

1) CLI wiring

A) Update `/tools/axion-control-plane/src/cli.ts`

1. Extend `Cmd`:

```
type Cmd =
  | "status"
  | "verify"
  | "run"
  | "init"
  | "ingest-results"
  | "emit"
  | "accept";
```

2. Add imports:

```
import { emitFromSpecGaps } from "./queue/emitFromSpecGaps.js";
import { acceptUnit } from "./queue/acceptUnit.js";
```

3. Add handlers (place near other command handlers):

```
 if (cmd === "emit") {
   const countArg = args.find((a) => a.startsWith("--count="));
   const count = countArg ? Number(countArg.split("=")[1]) : 1;

   const out = await emitFromSpecGaps(repoRoot, config, state, count);
   console.log(`Emitted PLAN_UNITs: ${out.emitted}`);
   if (out.emitted === 0) console.log("No eligible SPEC_INDEX gaps found.");
   return;
  }
```

```
  if (cmd === "accept") {
    const unitId = args[1];
    if (!unitId) {
      console.log("Usage: axion accept <UNIT_ID>");
      process.exit(1);
    }

    const out = await acceptUnit(repoRoot, config, unitId);
    console.log(`Accepted unit: ${unitId}`);
    console.log(`RESULT template: ${out.resultPath}`);
    return;
  }
```

---

2) `axion emit` implementation

Create `/tools/axion-control-plane/src/queue/emitFromSpecGaps.ts`

```
import path from "node:path";
import type { AxionConfig } from "../config.js";
import type { LoadedState } from "../state/loadState.js";
import { emitPlanUnit } from "./emitPlanUnit.js";

/**
 * Gap rule (MVP):
 * - target SPEC_INDEX items that are missing implementation_refs OR verification_proof
 * - prioritize by category order: endpoint, entity, screen, component, business_rule, flow,
test_case, acceptance_criterion
 * - emit up to `count` PLAN_UNITs (1 target per unit for simplicity)
 */
const CATEGORY_PRIORITY = [
  "endpoint",
  "entity",
  "screen",
  "component",
  "business_rule",
  "flow",
  "test_case",
  "acceptance_criterion"
] as const;
```

```
function missingAny(it: any): boolean {
  const implOk = Array.isArray(it?.implementation_refs) && it.implementation_refs.length > 0;
  const proofOk = Array.isArray(it?.verification_proof) && it.verification_proof.length > 0;
  return !(implOk && proofOk);
}

function alreadyQueuedOrInProgress(state: LoadedState, category: string, id: string): boolean {
  const key = `${category}:${id}`;
  const units = state.ledger.work_units ?? [];
  for (const wu of units) {
    if (!["queued", "in_progress"].includes(wu?.status)) continue;
    for (const t of wu?.targets ?? []) {
      if (`${t.category}:${t.id}` === key) return true;
    }
  }
  return false;
}

export async function emitFromSpecGaps(
  repoRoot: string,
  config: AxionConfig,
  state: LoadedState,
  count: number
): Promise<{ emitted: number; paths: string[] }> {
  const items = [...(state.specIndex.items ?? [])];

  // sort by category priority, then by id
  items.sort((a: any, b: any) => {
    const pa = CATEGORY_PRIORITY.indexOf(a.category);
    const pb = CATEGORY_PRIORITY.indexOf(b.category);
    const ca = pa === -1 ? 999 : pa;
    const cb = pb === -1 ? 999 : pb;
    if (ca !== cb) return ca - cb;
    return String(a.id).localeCompare(String(b.id));
  });

  const emittedPaths: string[] = [];
  let emitted = 0;

  for (const it of items) {
    if (emitted >= count) break;
    if (!it?.category || !it?.id) continue;
```

```
    if (!missingAny(it)) continue;
    if (alreadyQueuedOrInProgress(state, it.category, it.id)) continue;

    const unitId = `UNIT-${it.category.toUpperCase()}-${it.id}-${Date.now()}`;

    const unit = {
      schema_version: "1.0.0",
      unit_id: unitId,
      title: `Implement + verify ${it.category}:${it.id} — ${it.title}`,
      created_at: new Date().toISOString(),
      priority: 3,
      targets: [{ category: it.category, id: it.id }],
      acceptance: [
        `SPEC_INDEX item ${it.category}:${it.id} has non-empty implementation_refs`,
        `SPEC_INDEX item ${it.category}:${it.id} has non-empty verification_proof`,
        "axion verify passes"
      ],
      outputs_expected: [
        "code changes in repo",
        "RESULT_<UNIT_ID>.json with implementation_refs + proof_refs",
        "SPEC_INDEX updated for target with implementation_refs + verification_proof"
      ],
      notes:
        "Executor: implement the target, run verify commands, then write RESULT file referencing
commits/paths/logs."
    };

    const p = await emitPlanUnit(repoRoot, config, unit);

    // OPTIONAL: also add a queued work_unit entry to ledger now
    // (Keeping MVP minimal: ledger will be updated on accept/ingest.)
    emittedPaths.push(p);
    emitted++;
  }

  return { emitted, paths: emittedPaths };
}
```

---

3) `axion accept <UNIT_ID>` implementation

Create `/tools/axion-control-plane/src/queue/acceptUnit.ts`

```
import path from "node:path";
import fs from "node:fs/promises";
import type { AxionConfig } from "../config.js";
import { readJson, writeJson } from "../util/fs.js";

type PlanUnit = {
  schema_version: string;
  unit_id: string;
  title: string;
  created_at: string;
  targets?: Array<{ category: string; id: string }>;
};

export async function acceptUnit(
  repoRoot: string,
  config: AxionConfig,
  unitId: string
): Promise<{ resultPath: string }> {
  const queueDir = path.join(repoRoot, config.paths.queueDir);
  const ledgerPath = path.join(repoRoot, "axion/ledger.json");

  // Try to locate matching PLAN_UNIT in queue
  const plan = await findPlanUnit(queueDir, unitId);

  // Load ledger
  const ledger = (await readJson(ledgerPath)) as {
    schema_version: string;
    work_units: any[];
    proofs: any[];
  };

  const idx = ledger.work_units.findIndex((wu: any) => wu?.unit_id === unitId);

  if (idx === -1) {
    ledger.work_units.push({
      unit_id: unitId,
      title: plan?.title ?? `Work Unit ${unitId}`,
      status: "in_progress",
      created_at: plan?.created_at ?? new Date().toISOString(),
      completed_at: null,
      targets: plan?.targets ?? [],
      implementation_refs: [],
      verification_proof: [],
      notes: ""
```

```
    });
  } else {
    const wu = ledger.work_units[idx];
    wu.status = "in_progress";
    if (!wu.targets?.length && plan?.targets?.length) wu.targets = plan.targets;
  }

  await writeJson(ledgerPath, ledger);

  // Write RESULT template (executor fills)
  const resultTemplate = {
    schema_version: "1.0.0",
    unit_id: unitId,
    submitted_at: new Date().toISOString(),
    implementation_refs: [
      // examples:
      // "commit:<sha>",
      // "files:src/path/to/file.ts",
      // "pr:<number>"
    ],
    proof_refs: [
      // examples:
      // "runlog:axion/runs/RUN-.../test.log",
      // "runlog:axion/runs/RUN-.../build.log",
      // "manual:checked-xyz"
    ],
    summary: ""
  };

  const resultFile = `RESULT_${unitId}.json`;
  const resultPath = path.join(queueDir, resultFile);
  await writeJson(resultPath, resultTemplate);

  // Optional: move PLAN_UNIT into _accepted (keeps queue clean)
  if (plan?.__path) {
    const acceptedDir = path.join(queueDir, "_accepted");
    await fs.mkdir(acceptedDir, { recursive: true });
    await fs.rename(plan.__path, path.join(acceptedDir, path.basename(plan.__path)));
  }

  return { resultPath };
}
```

```
async function findPlanUnit(queueDir: string, unitId: string): Promise<(PlanUnit & { __path?:
string }) | null> {
  let entries: string[] = [];
  try {
    entries = await fs.readdir(queueDir);
  } catch {
    return null;
  }

  const candidates = entries
    .filter((f) => f.startsWith("PLAN_UNIT_") && f.endsWith(".json"))
    .map((f) => path.join(queueDir, f));

  for (const p of candidates) {
    try {
      const v = (await readJson(p)) as PlanUnit;
      if (v?.unit_id === unitId) return { ...v, __path: p };
    } catch {
      // ignore
    }
  }

  return null;
}
```

---

4) Usage

Emit work

node dist/bin.js emit --count=3

Accept a unit (creates RESULT template + marks in_progress)

node dist/bin.js accept UNIT-ENDPOINT-EP-0001-1700000000000

After executor fills RESULT + pushes code

node dist/bin.js ingest-results
node dist/bin.js verify

---

5) What this unlocks immediately

With your earlier gates (`GATE-LEDGER`, `GATE-TARGETS`), this creates a clean loop:

1. **emit** chooses the next spec gaps
2. **accept** creates the RESULT contract + marks unit active
3. executor does work + writes RESULT
4. **ingest-results** updates ledger (and, once you added the propagation, SPEC_INDEX too)
5. **verify** enforces that “done means proven + attached to targets”

If you want the next step, tell me whether your preferred unit granularity is **1 target per unit** (current) or **batch targets per unit** (faster throughput, slightly more complexity).

Rules

Locked: **1 target per unit** as the MVP rule.

Here's the minimal set of adjustments to make that rule *authoritative* and stable (no extra features, just correctness).

---

1) Enforce "exactly 1 target" everywhere

A) Update `/axion/schemas/plan_unit.schema.json`

Change `targets` to require **minItems/maxItems = 1**:

```
"targets": {
  "type": "array",
  "minItems": 1,
  "maxItems": 1,
  "items": {
    "type": "object",
    "required": ["category", "id"],
    "properties": {
      "category": { "type": "string", "enum":
["screen","endpoint","entity","business_rule","component","acceptance_criterion","test_case","flo
w"] },
      "id": { "type": "string" }
    }
  }
},
```

B) Update `/axion/schemas/ledger.schema.json`

Same thing for `work_units[].targets`:

```
"targets": {
  "type": "array",
  "minItems": 1,
  "maxItems": 1,
  "items": {
    "type": "object",
    "required": ["category", "id"],
    "properties": {
      "category": { "type": "string" },
```

```
      "id": { "type": "string" }
    },
    "additionalProperties": false
  }
},
```

---

2) Make `emit` only ever create 1-target units (already done)

Your `emitFromSpecGaps.ts` is already emitting `targets: [{...}]` only once. Keep it that way.

---

3) Make `accept` refuse units that violate the rule

Edit `/tools/axion-control-plane/src/queue/acceptUnit.ts` after loading `plan`:

Add:

```
if (plan?.targets && plan.targets.length !== 1) {
  throw new Error(`PLAN_UNIT ${unitId} violates 1-target rule
(targets.length=${plan.targets.length})`);
}
```

And when creating ledger work_unit, ensure targets is exactly one:

```
targets: plan?.targets?.slice(0, 1) ?? [],
```

---

4) Make `ledger_targets_implemented` assume 1 target (tighten output)

Since it's 1 target, your remediation can be cleaner, but functionally it's fine already. No change required.

---

5) Minimal “stable loop” command order (your operating rhythm)

This is the canonical cycle for 1-target units:

```
# 1) create next unit(s)
axion emit --count=1

# 2) pick a UNIT_ID and activate it
axion accept <UNIT_ID>

# 3) executor/agent does work, fills RESULT_<UNIT_ID>.json

# 4) ingest and attach refs
axion ingest-results

# 5) enforce gates
axion verify
```

---

6) One critical fix you should apply now (so TARGET gates can pass)

Your earlier `ingestResults.ts` upgrade that propagates refs to SPEC_INDEX depends on **ledger work_unit having targets**. With “1 target per unit,” that becomes deterministic.

So: ensure `acceptUnit.ts` always writes the plan unit’s single target into ledger. That’s the main stability anchor.

If you want, I can give you one more “stability” patch: a `axion pick` command that lists queued PLAN_UNITs and prints the next recommended UNIT_ID (so you don’t hunt filenames).

axion pick

Here's **`axion pick`** (lists available PLAN_UNITs in `/axion/queue`, shows the next recommended one, and prints the exact `axion accept <UNIT_ID>` command).

---

1) Wire the CLI command

Edit `/tools/axion-control-plane/src/cli.ts`

A) Add `pick` to Cmd

```
type Cmd =
  | "status"
  | "verify"
  | "run"
  | "init"
  | "ingest-results"
  | "emit"
  | "accept"
  | "pick";
```

B) Import

```
import { pickNextUnit } from "./queue/pickNextUnit.js";
```

C) Add handler

```
 if (cmd === "pick") {
    const out = await pickNextUnit(repoRoot, config);
    if (out.total === 0) {
      console.log("No PLAN_UNIT files found in queue.");
      return;
    }

    console.log(`PLAN_UNITs in queue: ${out.total}`);
    for (const row of out.list) {
      console.log(`- ${row.unit_id}  [${row.category}:${row.target_id}]  ${row.title}`);
    }

    if (out.recommended) {
      console.log("");
      console.log(`Recommended: ${out.recommended.unit_id}`);
```

```
    console.log(`Run: axion accept ${out.recommended.unit_id}`);
  }
  return;
}
```

---

2) Implement `pickNextUnit`

Create `/tools/axion-control-plane/src/queue/pickNextUnit.ts`

```
import path from "node:path";
import fs from "node:fs/promises";
import type { AxionConfig } from "../config.js";
import { readJson } from "../util/fs.js";

type PlanUnit = {
  schema_version: string;
  unit_id: string;
  title: string;
  created_at: string;
  priority?: number;
  targets: Array<{ category: string; id: string }>;
};

const CATEGORY_PRIORITY = [
  "endpoint",
  "entity",
  "screen",
  "component",
  "business_rule",
  "flow",
  "test_case",
  "acceptance_criterion"
] as const;

export async function pickNextUnit(
  repoRoot: string,
  config: AxionConfig
): Promise<{
  total: number;
  list: Array<{ unit_id: string; title: string; category: string; target_id: string; priority: number;
created_at: string }>;
```

```
  recommended: { unit_id: string; title: string; category: string; target_id: string } | null;
}> {
  const queueDir = path.join(repoRoot, config.paths.queueDir);

  let entries: string[] = [];
  try {
    entries = await fs.readdir(queueDir);
  } catch {
    return { total: 0, list: [], recommended: null };
  }

  const files = entries.filter((f) => f.startsWith("PLAN_UNIT_") && f.endsWith(".json"));

  const units: PlanUnit[] = [];
  for (const f of files) {
    const full = path.join(queueDir, f);
    try {
      const u = (await readJson(full)) as PlanUnit;
      if (!u?.unit_id || !u?.targets?.length) continue;

      // Enforce 1-target rule here too
      if (u.targets.length !== 1) continue;

      units.push(u);
    } catch {
      // ignore bad json
    }
  }

  // Sort: priority desc, category priority asc, created_at asc
  units.sort((a, b) => {
    const pa = a.priority ?? 0;
    const pb = b.priority ?? 0;
    if (pa !== pb) return pb - pa;

    const ca = catRank(a.targets[0].category);
    const cb = catRank(b.targets[0].category);
    if (ca !== cb) return ca - cb;

    return String(a.created_at).localeCompare(String(b.created_at));
  });

  const list = units.map((u) => ({
    unit_id: u.unit_id,
```

```
    title: u.title,
    category: u.targets[0].category,
    target_id: u.targets[0].id,
    priority: u.priority ?? 0,
    created_at: u.created_at
  }));

  const recommended = units.length
    ? {
        unit_id: units[0].unit_id,
        title: units[0].title,
        category: units[0].targets[0].category,
        target_id: units[0].targets[0].id
      }
    : null;

  return { total: units.length, list, recommended };
}

function catRank(category: string): number {
  const idx = (CATEGORY_PRIORITY as readonly string[]).indexOf(category);
  return idx === -1 ? 999 : idx;
}
```

---

3) Usage

axion pick

Output example:

- lists all queued PLAN_UNITs (1-target only)
- prints:
  `Run: axion accept <UNIT_ID>`

If you want one more stability add-on after this: `axion queue:clean` (moves malformed or multi-target plan units into `/axion/queue/_rejected/` so the queue stays deterministic).

axion queue:clean

Here's **`axion queue:clean`**. It scans `/axion/queue`, and moves anything that would cause drift/noise into deterministic buckets:

- invalid JSON → `_rejected/invalid_json/`
- not a PLAN_UNIT/RESULT file → `_rejected/unknown_files/`
- PLAN_UNIT violates **1-target rule** → `_rejected/multi_target/`
- PLAN_UNIT missing required fields → `_rejected/missing_fields/`

It **does not delete** anything.

---

1) Wire the CLI command

Edit `/tools/axion-control-plane/src/cli.ts`

A) Add command

```
type Cmd =
  | "status"
  | "verify"
  | "run"
  | "init"
  | "ingest-results"
  | "emit"
  | "accept"
  | "pick"
  | "queue:clean";
```

B) Import

```
import { cleanQueue } from "./queue/cleanQueue.js";
```

C) Handler

```
 if (cmd === "queue:clean") {
    const out = await cleanQueue(repoRoot, config);
    console.log(`Queue cleaned.`);
    console.log(`Moved: ${out.moved}`);
    for (const [k, v] of Object.entries(out.buckets)) {
      console.log(`- ${k}: ${v}`);
    }
```

```
    return;
  }
```

---

2) Implement `cleanQueue`

Create `/tools/axion-control-plane/src/queue/cleanQueue.ts`

```
import path from "node:path";
import fs from "node:fs/promises";
import type { AxionConfig } from "../config.js";
import { readJson } from "../util/fs.js";

type PlanUnit = {
  unit_id?: string;
  title?: string;
  created_at?: string;
  targets?: Array<{ category: string; id: string }>;
};

type ResultFile = {
  unit_id?: string;
  submitted_at?: string;
  implementation_refs?: string[];
  proof_refs?: string[];
};

export async function cleanQueue(
  repoRoot: string,
  config: AxionConfig
): Promise<{ moved: number; buckets: Record<string, number> }> {
  const queueDir = path.join(repoRoot, config.paths.queueDir);

  let entries: string[] = [];
  try {
    entries = await fs.readdir(queueDir);
  } catch {
    return { moved: 0, buckets: {} };
  }

  const buckets: Record<string, number> = {
    invalid_json: 0,
```

```
    unknown_files: 0,
    multi_target: 0,
    missing_fields: 0
  };

  let moved = 0;

  for (const name of entries) {
    // skip directories and our internal folders
    if (name === "_accepted" || name === "_processed" || name === "_rejected" || name ===
".gitkeep") continue;

    const full = path.join(queueDir, name);

    const stat = await safeStat(full);
    if (!stat || stat.isDirectory()) continue;

    const isPlan = name.startsWith("PLAN_UNIT_") && name.endsWith(".json");
    const isResult = name.startsWith("RESULT_") && name.endsWith(".json");

    if (!isPlan && !isResult) {
      await moveTo(queueDir, full, "unknown_files", name);
      buckets.unknown_files++;
      moved++;
      continue;
    }

    // Parse JSON
    let json: any;
    try {
      json = await readJson(full);
    } catch {
      await moveTo(queueDir, full, "invalid_json", name);
      buckets.invalid_json++;
      moved++;
      continue;
    }

    // Validate minimal required fields
    if (isPlan) {
      const plan = json as PlanUnit;

      const hasBasics = Boolean(plan.unit_id && plan.title && plan.created_at &&
Array.isArray(plan.targets));
```

```
      if (!hasBasics) {
        await moveTo(queueDir, full, "missing_fields", name);
        buckets.missing_fields++;
        moved++;
        continue;
      }

      // Enforce 1-target rule
      if ((plan.targets?.length ?? 0) !== 1) {
        await moveTo(queueDir, full, "multi_target", name);
        buckets.multi_target++;
        moved++;
        continue;
      }

      // Validate target fields
      const t = plan.targets![0];
      if (!t?.category || !t?.id) {
        await moveTo(queueDir, full, "missing_fields", name);
        buckets.missing_fields++;
        moved++;
        continue;
      }
    }

    if (isResult) {
      const res = json as ResultFile;
      const ok = Boolean(
        res.unit_id &&
          res.submitted_at &&
          Array.isArray(res.implementation_refs) &&
          Array.isArray(res.proof_refs)
      );
      if (!ok) {
        await moveTo(queueDir, full, "missing_fields", name);
        buckets.missing_fields++;
        moved++;
        continue;
      }
    }
  }

  return { moved, buckets };
}
```

```
async function safeStat(p: string) {
  try {
    return await fs.stat(p);
  } catch {
    return null;
  }
}

async function moveTo(queueDir: string, fullPath: string, bucket: string, filename: string) {
  const rejDir = path.join(queueDir, "_rejected", bucket);
  await fs.mkdir(rejDir, { recursive: true });
  const dest = path.join(rejDir, filename);

  // If dest exists, append timestamp
  const finalDest = await avoidCollision(dest);
  await fs.rename(fullPath, finalDest);
}

async function avoidCollision(dest: string): Promise<string> {
  try {
    await fs.stat(dest);
    const dir = path.dirname(dest);
    const base = path.basename(dest, ".json");
    return path.join(dir, `${base}.${Date.now()}.json`);
  } catch {
    return dest;
  }
}
```

---

3) Usage

axion queue:clean

This keeps the queue deterministic so `axion pick` and `axion accept` never choke on junk.

If you want the next stability move: add `axion queue:stats` (counts plan/result/accepted/processed/rejected + shows oldest unit age).

axion queue:stats

Here's **`axion queue:stats`**. It prints a deterministic summary of queue health:

- counts of `PLAN_UNIT_*` and `RESULT_*`
- counts in `_accepted`, `_processed`, `_rejected/*`
- oldest/newest PLAN_UNIT timestamps
- quick "next recommended" unit (same sorting as `pick`)

---

1) Wire the CLI command

Edit `/tools/axion-control-plane/src/cli.ts`

A) Add command

```
type Cmd =
  | "status"
  | "verify"
  | "run"
  | "init"
  | "ingest-results"
  | "emit"
  | "accept"
  | "pick"
  | "queue:clean"
  | "queue:stats";
```

B) Import

```
import { queueStats } from "./queue/queueStats.js";
```

C) Handler

```
 if (cmd === "queue:stats") {
    const out = await queueStats(repoRoot, config);
    console.log(`Queue Stats`);
    console.log(`- PLAN_UNIT in queue: ${out.planInQueue}`);
    console.log(`- RESULT in queue: ${out.resultInQueue}`);
    console.log(`- Accepted: ${out.accepted}`);
    console.log(`- Processed: ${out.processed}`);
    console.log(`- Rejected: ${out.rejectedTotal}`);
```

```
    if (out.rejectedByBucket) {
      for (const [bucket, n] of Object.entries(out.rejectedByBucket)) {
        console.log(`  - rejected/${bucket}: ${n}`);
      }
    }

    if (out.oldestPlanCreatedAt) console.log(`- Oldest PLAN created_at:
${out.oldestPlanCreatedAt}`);
    if (out.newestPlanCreatedAt) console.log(`- Newest PLAN created_at:
${out.newestPlanCreatedAt}`);

    if (out.recommended) {
      console.log(``);
      console.log(`Recommended next: ${out.recommended.unit_id}
[${out.recommended.category}:${out.recommended.target_id}]`);
      console.log(`Run: axion accept ${out.recommended.unit_id}`);
    }

    return;
  }
```

---

2) Implement `queueStats`

Create `/tools/axion-control-plane/src/queue/queueStats.ts`

```
import path from "node:path";
import fs from "node:fs/promises";
import type { AxionConfig } from "../config.js";
import { readJson } from "../util/fs.js";

type PlanUnit = {
  unit_id: string;
  title: string;
  created_at: string;
  priority?: number;
  targets: Array<{ category: string; id: string }>;
};

const CATEGORY_PRIORITY = [
  "endpoint",
  "entity",
```

```
  "screen",
  "component",
  "business_rule",
  "flow",
  "test_case",
  "acceptance_criterion"
] as const;

export async function queueStats(repoRoot: string, config: AxionConfig): Promise<{
  planInQueue: number;
  resultInQueue: number;
  accepted: number;
  processed: number;
  rejectedTotal: number;
  rejectedByBucket: Record<string, number>;
  oldestPlanCreatedAt: string | null;
  newestPlanCreatedAt: string | null;
  recommended: { unit_id: string; category: string; target_id: string } | null;
}> {
  const queueDir = path.join(repoRoot, config.paths.queueDir);

  const top = await safeList(queueDir);

  const planFiles = top.filter((f) => f.startsWith("PLAN_UNIT_") && f.endsWith(".json"));
  const resultFiles = top.filter((f) => f.startsWith("RESULT_") && f.endsWith(".json"));

  const accepted = (await safeList(path.join(queueDir, "_accepted"))).filter((f) =>
f.endsWith(".json")).length;
  const processed = (await safeList(path.join(queueDir, "_processed"))).filter((f) =>
f.endsWith(".json")).length;

  const rejectedByBucket: Record<string, number> = {};
  const rejectedDir = path.join(queueDir, "_rejected");
  const buckets = await safeList(rejectedDir);

  let rejectedTotal = 0;
  for (const b of buckets) {
    const n = (await safeList(path.join(rejectedDir, b))).filter((f) => f.endsWith(".json")).length;
    rejectedByBucket[b] = n;
    rejectedTotal += n;
  }

  // Load PLAN_UNITs (valid 1-target only) to compute oldest/newest and recommended
  const plans: PlanUnit[] = [];
```

```
  for (const f of planFiles) {
    try {
      const u = (await readJson(path.join(queueDir, f))) as PlanUnit;
      if (!u?.unit_id || !u?.created_at || !u?.targets?.length) continue;
      if (u.targets.length !== 1) continue;
      plans.push(u);
    } catch {
      // ignore parse errors (queue:clean can handle)
    }
  }

  plans.sort((a, b) => String(a.created_at).localeCompare(String(b.created_at)));
  const oldestPlanCreatedAt = plans.length ? plans[0].created_at : null;
  const newestPlanCreatedAt = plans.length ? plans[plans.length - 1].created_at : null;

  const recommended = pickRecommended(plans);

  return {
    planInQueue: planFiles.length,
    resultInQueue: resultFiles.length,
    accepted,
    processed,
    rejectedTotal,
    rejectedByBucket,
    oldestPlanCreatedAt,
    newestPlanCreatedAt,
    recommended
  };
}

function pickRecommended(plans: PlanUnit[]): { unit_id: string; category: string; target_id: string
} | null {
  if (plans.length === 0) return null;

  const sorted = [...plans].sort((a, b) => {
    const pa = a.priority ?? 0;
    const pb = b.priority ?? 0;
    if (pa !== pb) return pb - pa;

    const ca = catRank(a.targets[0].category);
    const cb = catRank(b.targets[0].category);
    if (ca !== cb) return ca - cb;

    return String(a.created_at).localeCompare(String(b.created_at));
```

```
  });

  const u = sorted[0];
  return { unit_id: u.unit_id, category: u.targets[0].category, target_id: u.targets[0].id };
}

function catRank(category: string): number {
  const idx = (CATEGORY_PRIORITY as readonly string[]).indexOf(category);
  return idx === -1 ? 999 : idx;
}

async function safeList(dir: string): Promise<string[]> {
  try {
    return await fs.readdir(dir);
  } catch {
    return [];
  }
}
```

---

3) Usage

axion queue:stats

If you want the next stability piece, the most valuable one is: **`axion doctor`** (checks for missing required `/axion/*` files, schema mismatches, and prints the exact fix commands).

axion doctor

Below is **`axion doctor`**. It does deterministic health checks and prints **actionable fixes** (paths + what to create). No magic, no side effects.

Checks:

- required `/axion/*` files exist
- required `/axion/*` folders exist
- JSON files parse
- quick schema sanity (minimal “required keys” checks)
- warns if queue contains invalid JSON (suggests `axion queue:clean`)

---

1) Wire the CLI command

Edit `/tools/axion-control-plane/src/cli.ts`

A) Add command

```
type Cmd =
  | "status"
  | "verify"
  | "run"
  | "init"
  | "ingest-results"
  | "emit"
  | "accept"
  | "pick"
  | "queue:clean"
  | "queue:stats"
  | "doctor";
```

B) Import

```
import { doctor } from "./doctor.js";
```

C) Handler

```
 if (cmd === "doctor") {
    const out = await doctor(repoRoot, config);
    console.log(`AXION Doctor`);
    console.log(out.ok ? "OK" : "ISSUES FOUND");
```

```
  for (const l of out.lines) console.log(l);

  if (!out.ok) process.exit(2);
  return;
}
```

---

2) Implement `doctor`

Create `/tools/axion-control-plane/src/doctor.ts`

```
import path from "node:path";
import fs from "node:fs/promises";
import type { AxionConfig } from "./config.js";
import { readJson } from "./util/fs.js";

type DoctorOut = { ok: boolean; lines: string[] };

export async function doctor(repoRoot: string, config: AxionConfig): Promise<DoctorOut> {
  const lines: string[] = [];
  let ok = true;

  // Required files
  const requiredFiles = [
    "axion/config.json",
    "axion/manifest.json",
    "axion/spec_index.json",
    "axion/ledger.json",
    "axion/gates.json",
    "axion/standards_index.json"
  ];

  // Required dirs
  const requiredDirs = ["axion/queue", "axion/runs", "axion/schemas"];

  lines.push("Checks:");

  for (const d of requiredDirs) {
    const exists = await existsDir(path.join(repoRoot, d));
    if (!exists) {
      ok = false;
```

```
      lines.push(`- FAIL dir missing: ${d}`);
      lines.push(`  Fix: mkdir -p ${d}`);
    } else {
      lines.push(`- PASS dir: ${d}`);
    }
  }

  for (const f of requiredFiles) {
    const exists = await existsFile(path.join(repoRoot, f));
    if (!exists) {
      ok = false;
      lines.push(`- FAIL file missing: ${f}`);
      lines.push(`  Fix: create ${f} (use your standard starter JSON)`);
    } else {
      lines.push(`- PASS file: ${f}`);
    }
  }

  // Parse JSON + minimal key checks
  ok = (await checkJson(repoRoot, "axion/config.json", ["project", "paths", "verifyCommands",
"gates"], lines)) && ok;
  ok = (await checkJson(repoRoot, "axion/manifest.json", ["schema_version", "project",
"generated_at", "artifacts", "last_run"], lines)) && ok;
  ok = (await checkJson(repoRoot, "axion/spec_index.json", ["schema_version", "items"], lines))
&& ok;
  ok = (await checkJson(repoRoot, "axion/ledger.json", ["schema_version", "work_units",
"proofs"], lines)) && ok;
  ok = (await checkJson(repoRoot, "axion/gates.json", ["schema_version", "definitions",
"history"], lines)) && ok;
  ok = (await checkJson(repoRoot, "axion/standards_index.json", ["schema_version",
"standards"], lines)) && ok;

  // Queue sanity: detect invalid json quickly
  const queueDir = path.join(repoRoot, config.paths.queueDir);
  const qWarn = await detectQueueInvalidJson(queueDir);
  if (qWarn > 0) {
    ok = false;
    lines.push(`- WARN queue has ${qWarn} invalid JSON file(s)`);
    lines.push(`  Fix: axion queue:clean`);
  } else {
    lines.push(`- PASS queue JSON parse`);
  }

  return { ok, lines };
```

```
}

async function checkJson(repoRoot: string, rel: string, requiredKeys: string[], lines: string[]):
Promise<boolean> {
  const full = path.join(repoRoot, rel);

  try {
    const v = await readJson(full);
    const missing = requiredKeys.filter((k) => !(k in v));
    if (missing.length) {
      lines.push(`- FAIL json keys: ${rel} missing [${missing.join(", ")}]`);
      lines.push(`  Fix: add required keys to ${rel}`);
      return false;
    }
    lines.push(`- PASS json parse + keys: ${rel}`);
    return true;
  } catch (e: any) {
    lines.push(`- FAIL json parse: ${rel}`);
    lines.push(`  Detail: ${e?.message ?? String(e)}`);
    lines.push(`  Fix: repair JSON or regenerate ${rel}`);
    return false;
  }
}

async function detectQueueInvalidJson(queueDir: string): Promise<number> {
  let entries: string[] = [];
  try {
    entries = await fs.readdir(queueDir);
  } catch {
    return 0;
  }

  const jsonFiles = entries.filter((f) => f.endsWith(".json"));
  let bad = 0;

  for (const f of jsonFiles) {
    const full = path.join(queueDir, f);
    try {
      const stat = await fs.stat(full);
      if (stat.isDirectory()) continue;
      await readJson(full);
    } catch {
      bad++;
    }
```

```
  }

  return bad;
}

async function existsFile(p: string): Promise<boolean> {
  try {
    const s = await fs.stat(p);
    return s.isFile();
  } catch {
    return false;
  }
}

async function existsDir(p: string): Promise<boolean> {
  try {
    const s = await fs.stat(p);
    return s.isDirectory();
  } catch {
    return false;
  }
}
```

---

3) Usage

axion doctor

- returns exit code 2 if issues found (CI-friendly)
- prints fixes (mkdir, recreate files, run queue:clean)

If you want the next “CI-ready” piece: a GitHub Action YAML that runs `axion doctor` + `axion verify` on every PR and blocks merges when gates fail.

Github-CI

Create this file in your repo:

`/.github/workflows/axion-control-plane.yml`

```yaml
name: AXION Control Plane

on:
  pull_request:
    types: [opened, synchronize, reopened, ready_for_review]
  push:
    branches: [main]

jobs:
  axion:
    name: axion doctor + verify
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "npm"
          cache-dependency-path: tools/axion-control-plane/package-lock.json

      - name: Install control plane deps
        working-directory: tools/axion-control-plane
        run: npm ci

      - name: Build control plane
        working-directory: tools/axion-control-plane
        run: npm run build

      - name: AXION Doctor (repo health)
        working-directory: tools/axion-control-plane
        run: node dist/bin.js doctor

      - name: AXION Verify (gates + commands)
        working-directory: tools/axion-control-plane
        run: node dist/bin.js verify
```

What this does

- On every PR and on pushes to `main`, it runs:
  - `axion doctor` (fails if the AXION folder/files/JSON are broken)
  - `axion verify` (runs your configured verifyCommands + gates)

One requirement

Make sure `tools/axion-control-plane/package-lock.json` exists (run `npm i` once and commit the lockfile), or remove the `cache-dependency-path` line.

If you want this to **block merges**, turn on branch protection in GitHub and require the “AXION Control Plane / axion doctor + verify” check to pass.